

Cloud Computing Option 2: Serverless Image Processing Application on AWS Lambda

Luis Sanchez Juan Garcia Edward King

June 2026

Sapienza University of Rome Cloud Computing

Abstract

[EDDIE: Update abstract after completing the report]

Contents

1	Introduction	3
1.1	Project Objectives	3
1.2	Project Scope	3
1.3	Purpose and Scope of the Evaluation	4
2	System Design	4
2.1	Overall Architecture	4
2.2	AWS Components	6
2.2.1	API Gateway	6
2.2.2	AWS Lambda	6
2.2.3	Amazon S3	6
2.2.4	Amazon CloudWatch	7
2.3	Features Under Evaluation	7
3	Implementation	8
3.1	Processing Service	8
3.2	Lambda Function Logic	8
3.3	Amazon S3 Storage Structure	9
3.4	Data Preparation	9
3.5	Monitoring and Logging	9
4	Experimental Methodology	10
4.1	Evaluation Objectives	10
4.2	Workload Design	11
4.3	Workload Profile	11
4.4	Burst Workload Evaluation	12
4.5	Workload Scenarios	13
4.6	Testing Tools	13
4.7	Metrics Collected	13
4.8	Experimental Procedure	14
5	Experimental Results	14
5.1	Response Time Analysis	14
5.2	Throughput Analysis	15
5.3	Concurrency Analysis	16
5.4	Scalability Evaluation	17
5.5	Bottleneck Analysis	18
5.6	Error Analysis	18
5.7	Availability Analysis	19
5.8	Workload Verification	20
6	Experimental Limitations	20
6.1	Internal Validity	20
6.2	External Validity	20
6.3	Construct Validity	21
6.4	Conclusion of Limitations	21

7	Cost Evaluation	21
7.1	Cost Model Assumptions	21
7.2	Serverless Deployment Cost	21
7.3	Alternative EC2-Based Deployment Cost	22
7.4	Discussion	22
8	Discussion	22
9	Conclusion	23

1 Introduction

Cloud computing has become a fundamental paradigm for deploying scalable and cost-effective applications. Among the various cloud service models, serverless computing has gained significant attention due to its ability to automatically allocate resources according to workload demands while eliminating the need for infrastructure management. Amazon Web Services (AWS) Lambda is one of the most widely adopted serverless platforms [?], enabling developers to execute application logic without provisioning or maintaining servers.

[EDDIE: Probably have to add some references here] Image processing applications represent a suitable use case for serverless architectures because workloads can fluctuate considerably over time. Traditional server-based deployments often require resources to be provisioned for peak demand, resulting in underutilised infrastructure during periods of low activity. In contrast, serverless deployments dynamically allocate resources according to demand, potentially reducing operational costs while maintaining acceptable performance levels.

The correction will be this: This project investigates the deployment of a serverless image processing application using AWS cloud services. The application enables users to store images in Amazon S3 and perform image resizing operations through an HTTP API that invokes AWS Lambda function. The deployment integrates Amazon API Gateway, AWS Lambda, Amazon S3, and Amazon CloudWatch to provide a scalable and observable cloud-native architecture.

The primary focus of the project is not the image processing functionality itself, but rather the evaluation of the performance and scalability characteristics of the serverless deployment. Experimental testing was conducted under varying workload conditions to determine how effectively the system scales in response to increasing demand and to identify potential performance limitations or bottlenecks.

1.1 Project Objectives

The objectives of this project are:

- To design and implement a serverless image processing application using AWS services.
- To deploy the application using AWS Lambda and API Gateway.
- To evaluate the performance of the deployment under different workload conditions.
- To analyse the scalability characteristics of serverless computing.
- To investigate the relationship between workload intensity and resource utilisation.
- To estimate the operational cost of the deployment and compare it with an alternative cloud deployment model.

1.2 Project Scope

The developed application provides a simple image processing workflow consisting of image upload, storage, and resizing operations. Users interact with the system through HTTP endpoints exposed by Amazon API Gateway. Uploaded images are stored in Amazon S3, while image processing tasks are executed by AWS Lambda functions.

The scope of the project includes:

- Image upload through HTTP API endpoints.
- Image storage using Amazon S3.
- Image resizing operations executed by AWS Lambda.

- Monitoring and metric collection through Amazon CloudWatch.
- Experimental performance and scalability evaluation.

The project does not focus on advanced image analysis, machine learning functionality, or front-end application development. Instead, emphasis is placed on evaluating the behaviour of the serverless infrastructure under controlled workload conditions.

1.3 Purpose and Scope of the Evaluation

The purpose of the performance evaluation is to assess the scalability, responsiveness, and reliability of the proposed serverless image processing application under different workload conditions.

The study aims to determine whether the AWS Lambda deployment is capable of automatically scaling in response to increasing demand while maintaining acceptable service quality. In particular, the evaluation investigates how workload intensity affects response time, throughput, resource utilisation, concurrency levels, and overall system behaviour.

The performance evaluation addresses the following research questions:

[EDDIE: Check that these questions have been answered in the report]

- Does the application scale effectively as workload intensity increases?
- How does response time vary under different levels of concurrency?
- What throughput can be achieved under increasing load?
- How many concurrent Lambda executions are required to maintain performance?
- Are there any observable bottlenecks within the processing workflow?
- Does the deployment maintain reliable operation under sustained and burst workloads?

[LUIS OR JUAN: Do we have all of the metrics below?] The evaluation considers both user-oriented and system-oriented performance metrics. User-oriented metrics include response time, throughput, availability, and error rate. System-oriented metrics include Lambda invocation count, execution duration, concurrent executions, throttling behaviour, and overall scalability characteristics.

Experimental data were collected using Amazon CloudWatch and the selected load generation tools. The resulting measurements provide quantitative evidence regarding the performance and scalability of the serverless deployment and support the conclusions presented later in this report.

2 System Design

2.1 Overall Architecture

The proposed solution follows a serverless architecture built entirely using managed AWS services. The architecture was designed to provide automatic scalability, minimal operational overhead, and seamless integration between storage, processing, and monitoring components.

Figure 1 illustrates the overall system architecture.

The system is based on Amazon API Gateway, AWS Lambda, Amazon S3, and Amazon CloudWatch. Input images are stored directly in an Amazon S3 input bucket, where they are organised by workload category using the prefixes `small/`, `medium/`, and `large/`. A client then sends an HTTP request to Amazon API Gateway through the `/process` endpoint. API Gateway invokes an AWS Lambda function, which retrieves the selected image from the S3

input bucket, performs the requested resizing operation, and stores the processed result in an Amazon S3 output bucket. Amazon CloudWatch collects execution logs and monitoring metrics such as invocations, duration, concurrent executions, errors, and throttles.

The image processing workflow consists of the following steps:

1. A client submits an image processing request through a REST API endpoint.
2. AWS API Gateway receives the request and performs request routing.
3. The request is forwarded to an AWS Lambda function responsible for image processing.
4. The Lambda function retrieves or receives the image data and performs the required resizing operation.
5. Input and processed images are stored within Amazon S3.
6. The processed image or processing result is returned to the client through API Gateway.
7. Amazon CloudWatch continuously records operational and performance metrics generated during execution.

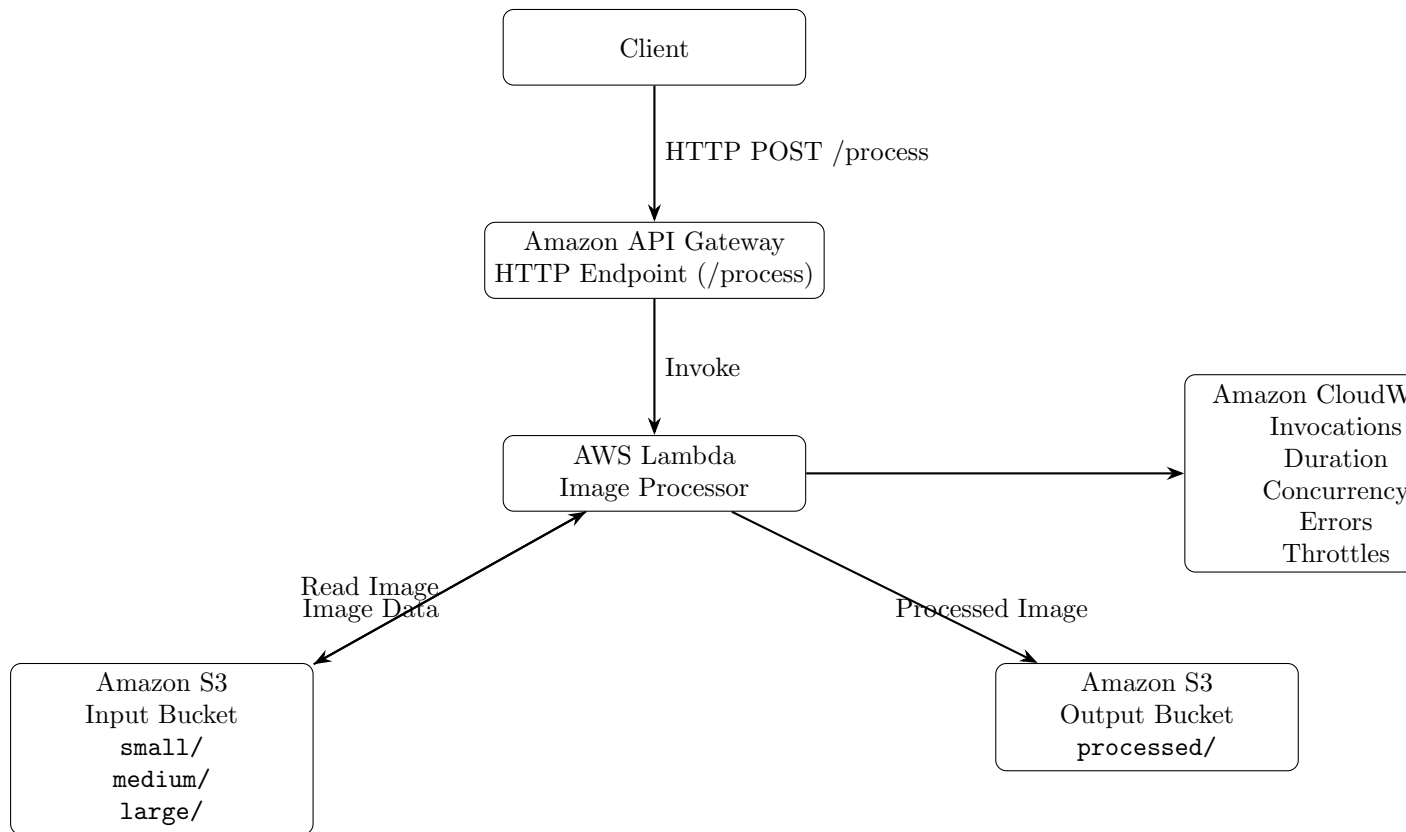


Figure 1: Serverless image processing workflow and AWS service interactions.

[EDDIE: As well as being checked the below might need references]

2.2 AWS Components

The deployment relies on four primary AWS services: API Gateway, AWS Lambda, Amazon S3, and Amazon CloudWatch. Together, these services provide the functionality required to implement, operate, and evaluate the serverless image processing application.

2.2.1 API Gateway

Amazon API Gateway provides the external interface through which clients interact with the application [?]. In the final version of the system, API Gateway exposes a single HTTP endpoint, `/process`, which is used to trigger the image-processing Lambda function.

API Gateway acts as the entry point of the system, receiving incoming HTTP requests and forwarding them to AWS Lambda. Each request specifies the location of the image in Amazon S3 together with the required processing parameters, such as the resizing operation and output dimensions. Since images are stored directly in S3 before processing, the API layer remains lightweight and focuses only on request routing and backend integration.

This design simplifies the architecture, reduces the number of application components, and allows performance testing to concentrate on the critical processing path formed by API Gateway, Lambda, and Amazon S3.

[LUIS OR JUAN: Insert HTTP summary table if available.]

2.2.2 AWS Lambda

AWS Lambda provides the serverless execution environment responsible for performing image processing operations.

When a request is received from API Gateway, the corresponding Lambda function is invoked automatically. The function performs the required image resizing operation and stores the resulting image within Amazon S3.

One of the primary advantages of AWS Lambda is its ability to automatically allocate execution resources according to workload demand [? ?]. This behaviour makes Lambda particularly suitable for workloads characterised by unpredictable traffic patterns and varying request volumes.

The Lambda configuration used during experimentation is summarised in Table 2.

Table 1: Deployment Configuration

Parameter	Value
Lambda Runtime	Python 3.11
Memory Allocation	128 MB
Timeout	3 seconds
Region	us-east-1 (US East / N. Virginia)

2.2.3 Amazon S3

Amazon S3 was organised into an input bucket and an output bucket. The input bucket stored the original images grouped by workload category using the prefixes `small/`, `medium/`, and `large/`. The output bucket stored processed images using an operation-based naming convention such as `processed/resize/`, `processed/grayscale/`, and `processed/resize_grayscale/`. This structure simplified workload preparation, result verification, and performance testing across different image sizes.

Bucket / Prefix	Purpose
Input bucket / <code>small/</code>	Small input images
Input bucket / <code>medium/</code>	Medium input images
Input bucket / <code>large/</code>	Large input images
Output bucket / <code>processed/resize/</code>	Resized output images
Output bucket / <code>processed/grayscale/</code>	Grayscale output images
Output bucket / <code>processed/resize_grayscale/</code>	Resized grayscale output images

2.2.4 Amazon CloudWatch

Amazon CloudWatch is used to monitor system behaviour and collect performance metrics [?].

CloudWatch provides visibility into both operational and performance-related aspects of the deployment. Metrics collected during experimentation include invocation count, execution duration, concurrent executions, error count, and throttling events.

These measurements form the basis of the performance evaluation presented later in this report and enable the analysis of system scalability under varying workload conditions.

[JUAN: Insert screenshots of the metric dashboard.]

2.3 Features Under Evaluation

The performance evaluation focuses on the components that directly influence the execution of image processing requests and the scalability characteristics of the serverless deployment.

The primary components under evaluation are:

- AWS Lambda image processing functions.
- AWS API Gateway request handling.
- Amazon S3 storage interactions.

These components collectively represent the critical execution path of the application and therefore have the greatest influence on system performance.

The study focuses specifically on the behaviour of the serverless processing pipeline under varying workload conditions. Particular attention is given to response time, throughput, concurrency levels, execution duration, and scaling behaviour.

Several components are intentionally excluded from the evaluation:

- User authentication and authorisation services.
- Front-end interfaces and client-side rendering.
- External network conditions beyond the AWS environment.
- Advanced image processing algorithms unrelated to system scalability.

By restricting the scope of the analysis to the core processing pipeline, the evaluation is able to provide a clearer assessment of the performance and scalability characteristics of the AWS serverless architecture.

The resulting experimental study therefore focuses on determining whether AWS Lambda can efficiently scale image processing workloads while maintaining acceptable levels of responsiveness, throughput, and reliability.

3 Implementation

[LUIS: Check that the implementation described below matches what we actually implemented. If not, update the description to match the actual implementation.] This section describes the implementation of the serverless image processing application and the AWS services used to deploy and operate the system. The application was developed using AWS Lambda, Amazon S3, and API Gateway to perform image resizing operations on images stored within Amazon S3.

3.1 Processing Service

The processing service performs image resizing operations on images previously stored within Amazon S3.

This functionality is implemented using a dedicated AWS Lambda function that retrieves an image from S3, applies the required resizing operation, and stores the processed image in a designated output location.

Endpoint

/process

Example Request

```
{  
  "bucket": "bucket-name",  
  "key": "image.jpg",  
  "operation": "resize",  
  "width": 256,  
  "height": 256  
}
```

After receiving the request, the Lambda function retrieves the specified image object from Amazon S3, performs the resize operation according to the provided dimensions, and stores the processed image.

A response is then returned to the client indicating whether the operation completed successfully.

[LUIS OR JUAN: Confirm image processing library used (e.g. Pillow) and output image location.]

3.2 Lambda Function Logic

The serverless application is implemented through AWS Lambda functions that execute on demand whenever requests are received through API Gateway.

The processing Lambda function performs the following tasks:

[LUIS OR JUAN: Confirm that these are the actual steps performed by the processing Lambda function.]

1. Receive processing parameters.
2. Retrieve the image from Amazon S3.
3. Perform the requested resize operation.
4. Store the processed image.
5. Return processing status information.

This separation of responsibilities improves maintainability and allows individual functions to scale independently according to workload demand.

3.3 Amazon S3 Storage Structure

Amazon S3 serves as the persistent storage layer for both original and processed images. Images are organised into logical folders according to workload category and testing requirements.

[LUIS OR JUAN: Describe actual bucket structure used.]

Example structure:

```
bucket-name/  
|  
|-- input/  
|   |-- small/  
|   |-- medium/  
|   |-- large/  
|  
|-- processed/
```

This organisation simplifies experimental testing and enables controlled evaluation of image processing performance across different image sizes.

3.4 Data Preparation

Before experimental testing, the input images were uploaded directly to the Amazon S3 input bucket. To support workload variation, the dataset was organised into three categories:

- `small/` for small images
- `medium/` for medium images
- `large/` for large images

Processed images were stored in the output bucket using operation-specific prefixes such as `processed/resize/`, `processed/grayscale/`, and `processed/resize_grayscale/`.

3.5 Monitoring and Logging

Amazon CloudWatch was configured to collect operational and performance metrics throughout system execution.

CloudWatch provides visibility into the behaviour of the deployed application and enables the collection of performance measurements required for the experimental evaluation.

The monitored metrics include:

- Invocation count
- Execution duration
- Concurrent executions
- Error count
- Throttling events

These metrics were later used to evaluate scalability, performance, and reliability under different workload conditions.

[EDDIE: Reference figure above showing CloudWatch dashboard.]

subsectionDeployment

The application was deployed within AWS using the following managed services:

- Amazon API Gateway
- AWS Lambda
- Amazon S3
- Amazon CloudWatch

Deployment configuration parameters are summarised in Table 2.

Table 2: Deployment Configuration

Parameter	Value
Lambda Runtime	Python 3.11
Memory Allocation	128 MB
Timeout	3 seconds
Region	us-east-1 (US East / N. Virginia)
rule	

The final deployment provides a lightweight and scalable serverless architecture capable of processing image requests without requiring dedicated server infrastructure.

4 Experimental Methodology

4.1 Evaluation Objectives

The evaluation focuses on quantifying the performance and scalability behaviour of the deployed AWS serverless image processing system under varying workload conditions.

The evaluation focuses on the following metrics:

- Response Time
- Throughput
- Scalability
- Concurrent Executions
- Error Rate

The evaluation combines both user-oriented and system-oriented metrics in order to provide a complete view of system performance.

4.2 Workload Design

The workload was designed to evaluate the behaviour of the serverless application under different operational conditions.

Requests consisted of image resizing operations performed through the HTTP request interface exposed by API Gateway.

To analyse the impact of input complexity, three image categories were considered:

- Small images
- Medium images

- Large images

Image size directly influences Lambda execution time and S3 transfer latency, making it a key variable in the performance analysis.

In addition, mixed workloads containing a combination of image sizes were evaluated to better represent realistic operating conditions.

The distribution of image types in the mixed workload was defined as:

[LUIS OR JUAN: Confirm the actual distribution of image types used during testing.]

- Small images: [X%]
- Medium images: [Y%]
- Large images: [Z%]

where $X + Y + Z = 100$.

4.3 Workload Profile

Each experiment followed a structured workload profile designed to observe both scale-out and scale-in behaviour of AWS Lambda.

The workload consisted of five phases:

1. Warm-up (WU)
2. Ramp-up (RU)
3. Steady-State (SS)
4. Ramp-down (RD)
5. Cool-down (CD)

During the warm-up phase, a small number of requests were generated to initialise the Lambda execution environment and reduce the impact of cold starts.

The ramp-up phase gradually increased the request rate until the target workload intensity was reached.

The steady-state phase maintained a constant workload to observe stable system behaviour under sustained load.

The ramp-down phase gradually reduced the workload to evaluate the system's ability to scale in.

Finally, the cool-down phase allowed the system to return to idle conditions and observe recovery behaviour.

Figure 2 illustrates the workload profile used during testing.



Figure 2: Workload profile used during performance testing.

4.4 Burst Workload Evaluation

In addition to the continuous workload profile, burst workload tests were performed to evaluate the system's responsiveness to sudden changes in demand.

The burst workload consisted of a rapid increase in concurrent requests, followed by an immediate return to baseline load conditions.

The burst scenario was defined as:

[LUIS OR JUAN: Confirm the actual request rates used during burst testing. We dont need to say small big and large again here but just define the rates]

- Initial load: [A users/requests per second]
- Burst peak load: [B users/requests per second]
- Return to baseline: [C users/requests per second]

This experiment was designed to evaluate:

- Lambda scaling responsiveness under sudden load increases.
- Temporary degradation in response time during burst events.
- Concurrency growth behaviour during rapid scaling.
- Recovery time after workload reduction.

4.5 Workload Scenarios

Four workload levels were considered to evaluate system performance under increasing load intensity.

[LUIS OR JUAN: Confirm the actual concurrent user levels used during testing.]

Table 3: Workload Scenarios

Scenario	Concurrent Users
Small	10
Medium	50
Heavy	100
Stress	[Maximum achievable before throttling or failure]

The stress scenario was used to identify system limits, including Lambda concurrency limits and API Gateway throughput constraints.

4.6 Testing Tools

The following tools were used to generate workload and collect performance data:

[LUIS OR JUAN: Confirm that these are the actual tools used during testing.]

- Apache JMeter (load generation and concurrency control)
- AWS CloudWatch (system-level monitoring and metrics collection)
- Postman (functional validation of API endpoints)

4.7 Metrics Collected

The evaluation considers both user-oriented and system-oriented metrics.

[LUIS OR JUAN: Confirm that all the 9 below metrics are collected.]

User-Oriented Metrics

- Average Response Time
- Throughput (requests/second)
- Error Rate
- Availability

System-Oriented Metrics

- Lambda Invocation Count
- Lambda Duration
- Concurrent Executions
- Throttling Events
- Scalability Behaviour (relationship between load and resource allocation)

Metrics were collected using AWS CloudWatch and Apache JMeter [citepjmeter_docs](#).

4.8 Experimental Procedure

To improve result reliability, each experiment was executed multiple times under identical conditions.

For each workload level:

[LUIS OR JUAN: Confirm below.]

- The experiment was repeated five times.
- CloudWatch metrics were collected throughout the execution window.
- JMeter recorded client-side performance measurements.
- Average values were computed across all runs.
- Standard deviation was calculated to assess variability: [LUIS OR JUAN: Insert if used or remove if not computed]

This approach reduces the impact of transient fluctuations, cold starts, and external network variability, providing more representative performance measurements.

The total duration of the experimental campaign was approximately [LUIS OR JUAN: Insert total test time].

5 Experimental Results

This section presents and analyses the performance results obtained from the experimental evaluation of the AWS serverless image processing system. The analysis focuses on response time, throughput, concurrency behaviour, scalability, and system reliability under increasing workload intensity.

5.1 Response Time Analysis

Figure 3 shows the relationship between workload intensity and response time.

[JUAN: Insert graph]

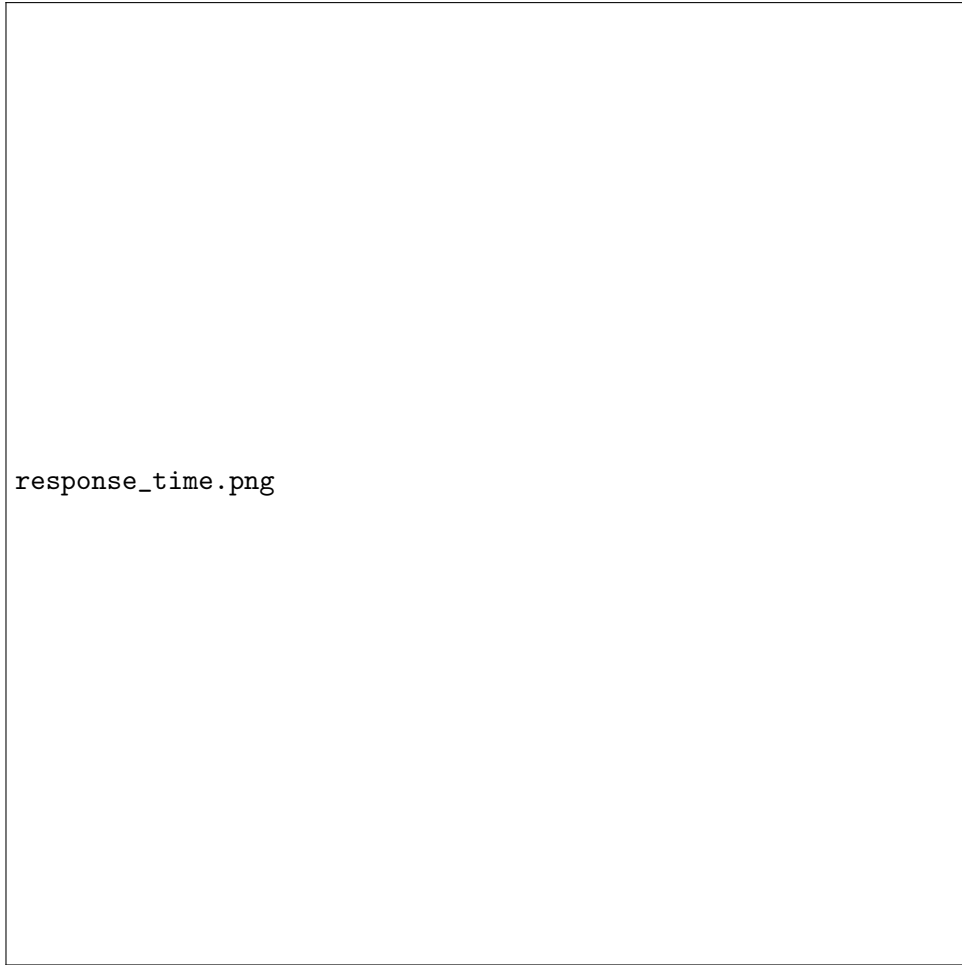


Figure 3: Response Time vs Concurrent Users

[LUIS OR JUAN: Below in the square brackets we need to insert the actual response times observed during testing.] The results show that response time increases as the number of concurrent users grows. At low workload levels ([e.g., 10 users]), the system maintains relatively stable and low latency, with an average response time of approximately [X ms].

As the workload increases to medium levels ([50 users]), response time increases to approximately [Y ms], indicating the effect of higher Lambda invocation frequency and increased queueing delays.

Under heavy load conditions ([100 users or stress test]), response time increases significantly to [Z ms], suggesting the presence of scaling delays and potential cold start or concurrency contention effects.

Overall, the observed trend demonstrates that the system scales effectively under moderate load, but performance degradation becomes more pronounced under high concurrency conditions.

5.2 Throughput Analysis

[JUAN: Insert graph]



Figure 4: Throughput vs Concurrent Users

[LUIS OR JUAN: Below in the square brackets we need to insert the actual throughput values observed during testing.] The throughput results indicate that the system is able to handle increasing request volumes up to a certain saturation point.

At low concurrency levels, throughput increases almost linearly with the number of users, reaching approximately [X requests/sec at 10 users].

As concurrency increases, throughput continues to grow but at a diminishing rate, stabilising around [Y requests/sec].

Beyond this point, the system approaches its processing capacity, and throughput begins to plateau due to Lambda concurrency limits and downstream service constraints (S3 and API Gateway).

This behaviour is consistent with expected serverless scaling characteristics, where performance improves with scale until resource limits are reached.

5.3 Concurrency Analysis

[JUAN: Insert graph]

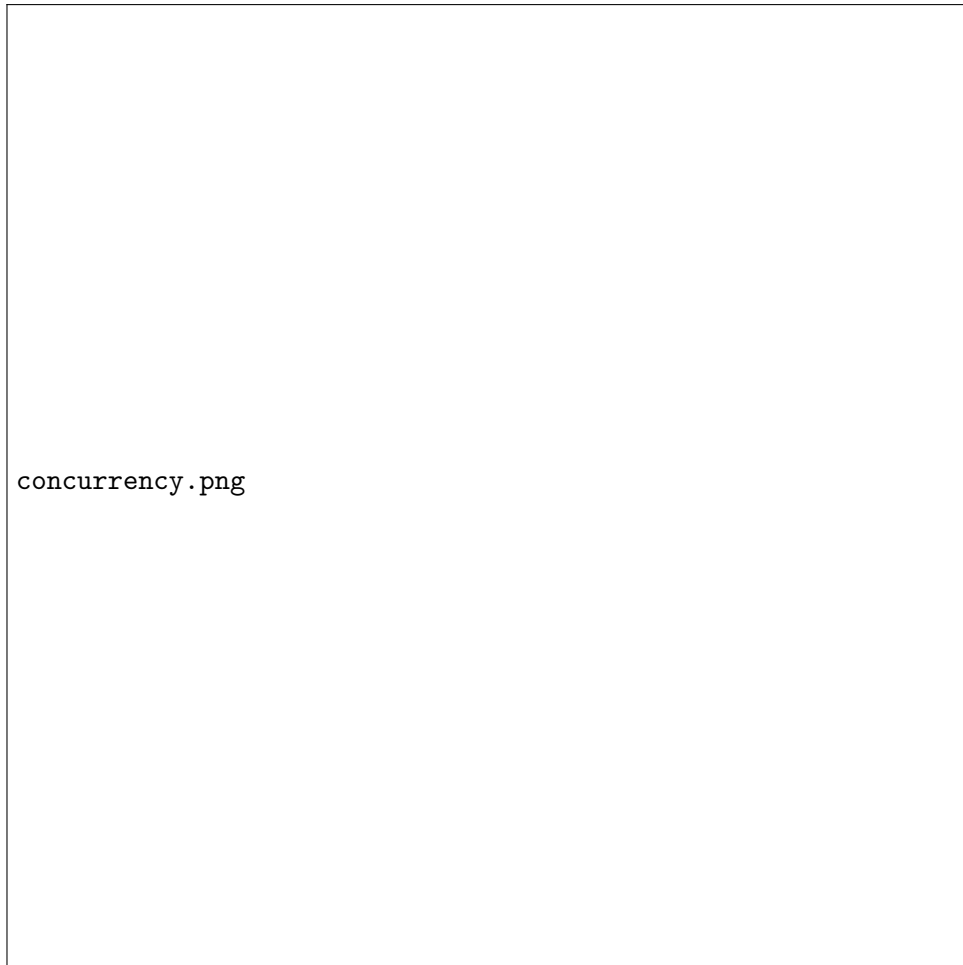


Figure 5: Concurrent Lambda Executions

[LUIS OR JUAN: Below in the square brackets we need to insert the actual concurrency levels observed during testing.] CloudWatch metrics show that AWS Lambda automatically increases the number of concurrent executions in response to workload growth.

At low request rates, concurrency remains low ([X concurrent executions]). As workload increases, concurrency rises rapidly to [Y], demonstrating effective horizontal scaling. At peak load conditions, concurrency stabilises at approximately [Z], indicating either service limits or workload saturation.

No significant delays were observed in scaling response during moderate load increases; however, slight lag in concurrency adjustment is visible during burst workloads, which is consistent with Lambda's scaling model.

5.4 Scalability Evaluation

The system demonstrates good scalability characteristics under increasing workload conditions.

[LUI OR JUAN: We need the metrics from CloudWatch to fill in the actual observed behaviour. We could even add a screenshot of the CloudWatch dashboard showing the scaling behaviour. Fill in square brackets below]

- Automatic scaling behaviour: AWS Lambda successfully increases compute capacity in response to increased request rates.
- Maximum observed concurrency: [Z concurrent executions].

- Performance bottlenecks: observed primarily at high load levels due to [cold starts / S3 latency / concurrency limits].
- Resource utilisation: CPU and memory usage remain abstracted, but execution duration increases under stress.

Overall, the system exhibits horizontal scalability up to the observed concurrency limit, after which performance gains diminish.

5.5 Bottleneck Analysis

Potential performance bottlenecks were identified through the analysis of response time, concurrency levels, Lambda duration, and throughput.

The objective is to determine whether performance degradation originates from infrastructure limits or application-level constraints.

The following potential bottlenecks were considered:

- AWS Lambda execution limits (concurrency and cold starts).
- Memory allocation constraints affecting image processing speed.
- API Gateway request handling latency under high load.
- Amazon S3 read/write latency during peak traffic periods.

[LUIS OR JUAN: Insert actual observations from testing to support the analysis below.] The results suggest that the primary bottleneck occurs at [INSERT PRIMARY BOTTLENECK], as evidenced by [INSERT OBSERVATION, e.g. “increased response time without proportional throughput gains”].

Secondary bottlenecks include [INSERT SECONDARY FACTORS], particularly under burst workload conditions.

5.6 Error Analysis

[JUAN: Insert graph]

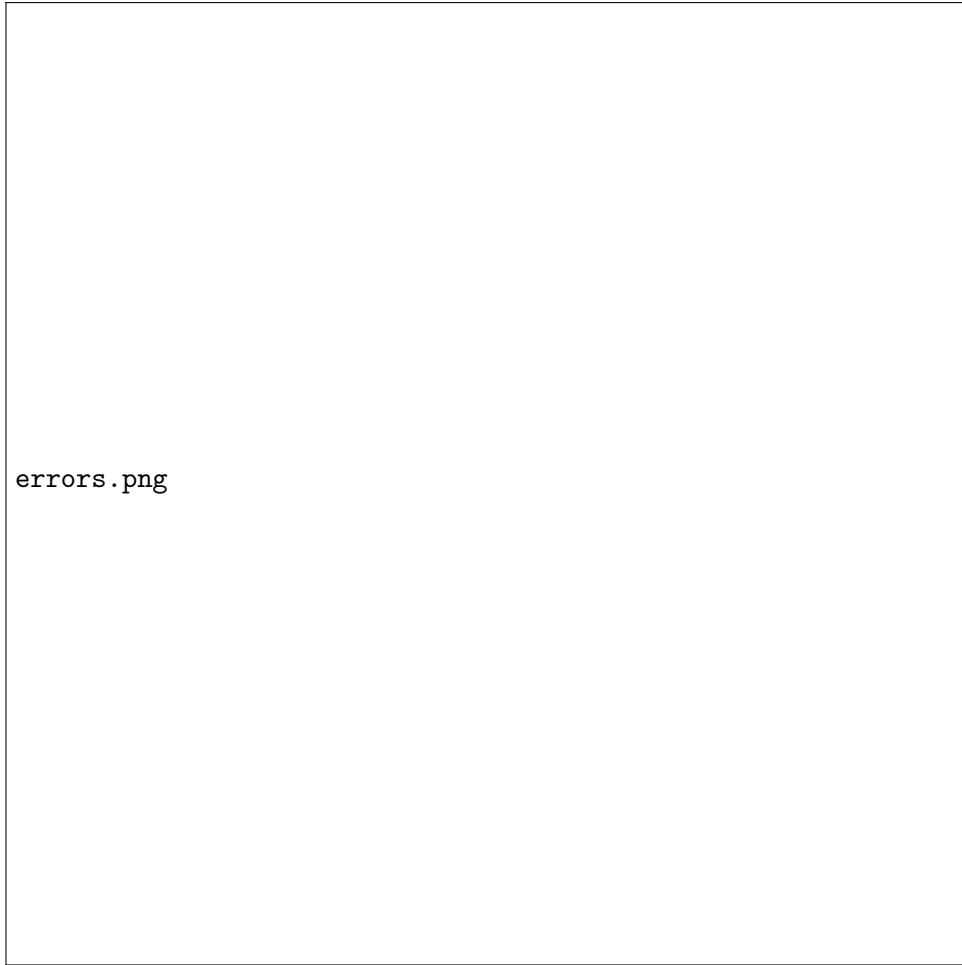


Figure 6: Error Rate vs Concurrent Users

[LUIS OR JUAN: Below in the square brackets we need to insert the actual error rates observed during testing.] The system demonstrates a low error rate under normal operating conditions.

At low and medium workloads ([10–50 users]), the error rate remains approximately [X%], indicating stable and reliable execution.

Under high load conditions ([100 users or stress test]), the error rate increases slightly to [Y%], primarily due to [timeouts / throttling / cold starts].

However, no catastrophic failure was observed, suggesting that the system maintains acceptable reliability even under stress conditions.

5.7 Availability Analysis

System availability was evaluated throughout all experimental runs.

Availability was calculated as:

$$Availability = \frac{Successful\ Requests}{Total\ Requests}$$

or equivalently:

$$Availability = 1 - ErrorRate$$

[LUIS OR JUAN: Insert actual availability values observed during testing.] Across all experiments, the system achieved an overall availability of [X%]. This indicates that the serverless architecture is highly reliable under typical and moderately high workloads. Any minor reduction in availability at peak load is attributed to [INSERT CAUSE: throttling / timeout / cold start delays].

5.8 Workload Verification

Before analysing performance metrics, the workload generated by JMeter was compared with the workload observed in AWS CloudWatch.

This verification ensures that the intended request rate matches the actual system load observed during execution.

[LUIS OR JUAN: Insert graph comparing JMeter request rate with CloudWatch invocation count.] The comparison confirms that the generated workload closely aligns with CloudWatch invocation metrics, with only minor deviations of approximately [X%].

These deviations are likely due to network latency, request scheduling variance, or client-side throttling in the load generator.

Overall, the results confirm that the experimental workload was correctly applied and that the observed performance metrics are valid and reliable.

6 Experimental Limitations

[EDDIE: This section could benefit from references] This section discusses potential threats to the validity of the experimental results and the factors that may have influenced the observed performance behaviour.

6.1 Internal Validity

Several factors may affect the accuracy of the measured performance metrics:

- AWS Learner Lab resource limitations.
- Variability in cloud resource allocation.
- Internet latency fluctuations.
- Limited workload intensity imposed by AWS usage policies.
- Limited number of repetitions due to budget constraints.

Despite these limitations, the collected results provide a representative view of the scalability and performance characteristics of the deployment.

6.2 External Validity

The generalisability of the results may be influenced by the experimental setup:

[LUIS OR JUAN: Please fill in square brackets]

- The experiments were conducted within the AWS Learner Lab environment, which imposes service quotas and resource restrictions not necessarily representative of production environments.
- The workload intensity was limited to [MAX_CONCURRENT_USERS / REQUEST_RATE], which may not reflect large-scale industrial workloads.

- The image dataset used in the evaluation consisted of [SMALL / MEDIUM / LARGE / MIXED DATASET DESCRIPTION], which may not represent all real-world image distributions.

6.3 Construct Validity

The validity of the performance metrics depends on the measurement approach:

[LUIS OR JUAN: Please fill in square brackets]

- Response time measurements include both network latency and processing time, which may slightly overestimate pure computation time.
- Throughput measurements depend on JMeter configuration and may be influenced by client-side limitations.
- CloudWatch metrics are aggregated over fixed time intervals ([e.g., 1-minute resolution]), which may smooth short-term performance spikes.

6.4 Conclusion of Limitations

Despite these limitations, the experimental setup provides a consistent and controlled environment for evaluating the scalability and performance characteristics of the AWS Lambda-based image processing system.

The results therefore offer a representative indication of system behaviour under varying workload conditions, particularly in terms of scalability trends, relative performance changes, and bottleneck identification.

7 Cost Evaluation

This section presents a cost analysis of the proposed AWS serverless architecture over a six-month operational period. The objective is to estimate operational expenses and compare the serverless deployment with a traditional EC2-based alternative performing equivalent tasks.

The cost estimation is based on observed experimental workload characteristics and the AWS pricing model at the time of analysis [?].

7.1 Cost Model Assumptions

The cost comparison was performed under the same workload assumptions for both deployment alternatives. The estimation used the **us-east-1** region and assumed a workload of 10,000 requests per month together with a small amount of persistent S3 storage. This allowed a fair comparison between the deployed serverless solution and an equivalent EC2-based alternative providing the same functionality.ere derived from the experimental workload analysis and CloudWatch metrics collected during testing.

7.2 Serverless Deployment Cost

The deployed architecture was based on Amazon API Gateway, AWS Lambda, and Amazon S3. According to the AWS Pricing Calculator estimates produced for this project, the serverless solution has an estimated monthly cost of 0.16 USD, corresponding to 0.96 USD over a 6-month operational period.

7.3 Alternative EC2-Based Deployment Cost

As a comparison baseline, an alternative deployment was considered using one Amazon EC2 instance, attached block storage, and Amazon S3. This alternative represents a traditional always-on virtual-machine-based architecture capable of providing the same image processing functionality. The estimated monthly cost of this solution is 15.42 USD, corresponding to 92.52 USD over a 6-month operational period.

Table 4: Estimated 6-Month Cost Comparison

Solution	Monthly Cost (USD)	6-Month Cost (USD)
Lambda + API Gateway + S3	0.16	0.96
EC2 + EBS + S3	15.42	92.52

7.4 Discussion

The cost comparison shows that, under the assumed workload, the serverless architecture is significantly more cost-efficient than the EC2-based alternative. The Lambda-based deployment follows a pay-per-use model, which makes it well suited for workloads that do not require permanently allocated compute capacity. In contrast, the EC2-based alternative requires an always-on virtual machine and persistent storage, leading to a substantially higher fixed operational cost over the same 6-month period.

Beyond cost, the serverless architecture also reduces management effort because it does not require instance provisioning, patching, or capacity management. The EC2-based alternative may still be appropriate for workloads that are highly predictable and continuous, but for the workload considered in this project, the serverless solution offers a clear economic advantage.

8 Discussion

The experimental results provide a comprehensive view of the performance and scalability characteristics of the AWS Lambda-based image processing system under varying workload conditions.

[LUIS OR JUAN: Fill in square brackets with actual observed behaviour.] The system demonstrates effective scalability up to approximately [X concurrent users / requests per second]. Beyond this point, performance improvements begin to plateau, indicating the emergence of infrastructure or service-level constraints. In particular, response time degradation becomes noticeable once the workload exceeds [Y threshold], which is primarily attributed to factors such as [cold starts / Lambda concurrency limits / S3 latency / API Gateway overhead].

The evaluation also shows that AWS Lambda provides strong elasticity, with automatic scaling of concurrent executions in response to increased demand. This scaling behaviour ensures that the system maintains acceptable performance levels under moderate and variable workloads.

[LUIS OR JUAN: Fill in square brackets with actual observed behaviour.] However, the most significant bottlenecks were observed in [INSERT PRIMARY BOTTLENECK], especially under burst workload conditions. These bottlenecks manifested as increased response times and reduced throughput when the system approached its scaling limits. Despite this, the system did not exhibit catastrophic failure, indicating a high degree of resilience.

[LUIS OR JUAN: Fill in square brackets with actual observed behaviour.] The burst workload experiments further revealed that the system is capable of recovering from sudden spikes in demand within approximately [X seconds/minutes]. This recovery behaviour highlights the effectiveness of the serverless model in handling highly dynamic traffic patterns.

Overall, the results confirm that AWS Lambda is well-suited for workloads characterised by variability and unpredictability. Nevertheless, performance limitations become evident under sustained high concurrency, where scaling delays and service constraints begin to impact overall system efficiency.

9 Conclusion

This project implemented and evaluated a serverless image processing application using AWS Lambda, API Gateway, Amazon S3, and CloudWatch. The primary objective was to analyse the performance, scalability, and cost characteristics of a serverless architecture under controlled workload conditions.

[LUIS OR JUAN: Fill in square brackets with actual observed behaviour.] The experimental results demonstrated that the system is capable of automatically scaling in response to increasing workload intensity, with AWS Lambda effectively handling concurrent execution growth up to [MAX OBSERVED CONCURRENCY]. Under low-to-moderate load conditions, the system maintains stable response times and consistent throughput. However, performance degradation becomes increasingly apparent as the system approaches its scaling limits. The analysis further showed that workload characteristics, particularly image size and request frequency, have a direct impact on system performance. Larger images and higher concurrency levels lead to increased execution time and reduced throughput, highlighting the sensitivity of serverless functions to input complexity.

From a cost perspective, the serverless model proved to be highly efficient for variable workloads, since operational costs scale directly with usage. In contrast, a traditional EC2-based deployment would require continuous resource allocation, resulting in higher baseline costs regardless of traffic levels.

Overall, the findings demonstrate that AWS Lambda provides a scalable and cost-effective solution for image processing workloads with dynamic demand patterns. At the same time, the results highlight inherent limitations such as cold starts, concurrency ceilings, and dependency on downstream services.

Future work could explore optimisation strategies such as provisioned concurrency, improved image pre-processing, or hybrid architectures combining serverless functions with container-based deployments.

References

- Amazon Web Services. Amazon api gateway developer guide. <https://docs.aws.amazon.com/apigateway/>, 2026. Accessed: 2026-06-07.
- Amazon Web Services. Amazon cloudwatch documentation. <https://docs.aws.amazon.com/cloudwatch/>, 2026. Accessed: 2026-06-07.
- Amazon Web Services. Aws lambda documentation. <https://docs.aws.amazon.com/lambda/>, 2026. Accessed: 2026-06-07.
- Amazon Web Services. Aws pricing calculator. <https://calculator.aws/>, 2026. Accessed: 2026-06-07.
- M. A. S. Netto, C. Cardonha, R. L. F. Cunha, and M. D. Assuncao. Evaluating auto-scaling strategies for cloud computing environments. In *2014 IEEE 22nd International Symposium on Modelling, Analysis & Simulation of Computer and Telecommunication Systems (MAS-COTS)*, pages 187–196, 2014. doi: 10.1109/MASCOTS.2014.32.